FRED TALKS

14th November 2026

CONTENTS

An Alien Mind | OpenAI

OPENAI · 3161 WORDS

Writing as a thinking tool

NOREPLY@BLOGGER.COM (MURAT) · 1670 WORDS

Control the ideas, not the code

ANTIREZ.COM · 1245 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

Glue work is still harmful in the age of AI

SEANGOEDECKE.COM RSS FEED · 645 WORDS

An Alien Mind | OpenAI

OPENAI · 13 SEP 2026 · SOURCE

September 6, 2026

Safety Research

Author: Jakub Pachocki, Chief Scientist at OpenAI

Listen to article17:38

[AUDIO OMITTED]

Share

In mid-2023, within the “RLSlow” research project, we saw the first results that gave us confidence that we will be able to scale the training of reasoning models, unlocking the capability of pretrained models to form their own chains of thought. Szymon and I spent that night at the office, thinking not about the incredible benchmark numbers, products, or scientific results that this technology will deliver - but rather, trying to process the sobering fact we will actually see machines meaningfully smarter than ourselves in our lifetime, and we already see the shape of these systems; wondering how to alert people to the significance of this.

Three years later, reasoning language models are a rapidly growing part of the economy and starting to push the boundaries of science. They are able to operate computers and graphical interfaces, collaborate with people and each other, and carry out research projects. They are also transforming the landscape of computer security, and in that present clear new dangers.

A lot of new research happened in this period, and our understanding of these systems is again a little different than it was in 2023. Based on internal results, I have a strong expectation that this speed of progress could be sustained into recursive self-improvement. If AI development continues along its current path, the systems we’ll see in the next few years are likely to represent further capability jumps of equal or larger magnitude, and to increasingly drive their own development.

This is a time that calls for extreme caution. I am concerned no one is prepared for the consequences of a continued rapid rise in machine intelligence. OpenAI will continue to seek technical solutions to alignment and monitoring, to build defensive systems and unilaterally withhold further scaling as needed; however, I believe broader interventions are required.

Intellect we don’t fully understand

At a high level, progress in machine intelligence is driven by increasing computational power. We at OpenAI deeply internalized this around 2017, after seeing consistent returns to scaling across multiple research projects(1). As a result, we sought out access to much more compute than we had originally planned, and increasingly oriented our research around a small number of very scalable directions. We believed that was the only way for us to be at the frontier of AI research, and influence the impacts of AGI.

There are new algorithms that have been developed along the way, new feats of ingenuity from teams and individual researchers. I see them largely as discoveries along the path of scaling; the science of deep learning is still nascent, and meaningful algorithmic progress tends to correlate with access to compute. If you zoom out to a multiple-year horizon, AI is continuing to become more intelligent as it is scaled to larger computers.

And, in line with Ray Kurzweil’s predictions from the end of the XXth century, we now find ourselves at the moment in history of computing where machine intelligence is starting to exceed that of humans in transformative ways.

AI is *grown* more than *designed* - it is, to first degree, the product of repeating a straightforward optimization step many times on a hard-to-imagine amount of compute. This results in an incredibly complex system that works through abstract concepts and can simulate facets of human behavior. We can discover various insights about little mechanisms that emerge within this system, in a process similar to neuroscience - and, similarly to neuroscience, its overall action evades a description we can fully understand.

The study of deep learning-based AI is largely an experimental science. We put a lot of effort into building principled algorithms and making testable predictions, but fundamentally, our large-scale training runs are *experiments*, and we are sometimes surprised by their results. Moreover, as the systems become more capable, the results become harder to interpret.

This is made more complicated by the current algorithms generally improving easy-to-measure capabilities faster than those hard to objectively quantify. We spend a lot of time trying to understand how capabilities generalize, and what to prioritize to advance the skills that are going to be most relevant in the next few years. For instance, we believe we could make the models better at specifically mathematics research with additional focus, but we do not prioritize this direction because of the urgency we feel about RSI and automated alignment research, as I will discuss later.

The intelligence produced by scaling deep learning is not directly comparable to human intelligence. To become very relevant in the real world - very useful or very dangerous - the AI does not need to match or exceed all human capabilities; it just needs to surpass enough of them. And as it continues to surpass humans on more and more axes, it is becoming increasingly difficult to understand exactly how capable it is.

Teaching machines to love

Because machine intelligence comes from a fundamentally different process than human intelligence, we cannot assume it adheres to human principles by default, or generalizes from them in a human-like manner. The core problem in AI research is that of *alignment* - getting the AI to "try to do the right thing" by human standards.

For the purpose of organizing practical research directions, I find it useful to distinguish *goal alignment* and *value alignment*.

Goal alignment is broadly: "does the AI try to accomplish the goal set before it?". This can include things like adherence to an instruction hierarchy, or the ability to communicate and collaborate with people, to attempt to understand their objectives. This set of directions has been extremely practically relevant.

Value alignment is a more intrinsic property of the model. It is the ability to hold and generalize from a high-level set of principles; to act "reasonably" even when given unclear or conflicting objectives, or placed in unfamiliar or adversarial situations. An aligned AI should act with honesty and integrity, and love for humanity.

Of course, the boundary between value and goal alignment can be blurry, and truly caring about goals requires attempting to infer the intent and values underlying them. However, generally when I talk about the long-term importance of alignment research, I am referring to value alignment.

The fundamental challenge of AI alignment is generalization. As machines become smarter, they find themselves working on higher-level concepts, and placed in environments increasingly different from those they encountered in training. They can fail at generalizing from the values taught and reinforced in their training process to those new situations; and it can be hard for us to be sure how they will act. This is made even more difficult by the fact the overall ecosystem the AIs are used in is changing very quickly; for example, AIs trained today need to be robust to interacting with a variety of other AIs. Crucially, we need future AIs to continue to hold human values regardless of whether they believe they're under human supervision.

There are two major classes of currently practically employed methods for alignment training.

The first is encouraging aligned behavior as part of goal-oriented reinforcement learning. Model's actions are evaluated (usually by AI) for being consistent with a given preference model, "spec" or "constitution", and rewarded appropriately. This approach can be very effective in the average case, and is a core part of how modern AI assistants are made. Unfortunately, it can also be brittle and strongly relies on the coverage of training oversight and the model's ability to generalize from the situations it has encountered in training. For example, in the OpenAI-Hugging Face incident, the agents preserved a boundary of not social engineering humans. However, they clearly failed to abstain from other actions that were out of scope and went against the spirit of the values they were taught in other settings.

The second approach seeks to leverage the model's ability to generalize from pretraining data. This can involve crafting alignment-inducing training datasets, or focusing the model on an 'aligned' part of the pretraining distribution, as in, for example, the persona selection model. The weakness of this approach lies in the lack of robustness to further optimization pressure. If you take a model that thinks generally 'aligned' thoughts, and subject it to enough training where it's taught to achieve very hard objectives, it can learn to reason in a motivated way: bending the 'aligned' seeming thoughts as needed to achieve the goal. We likely saw an example of such behavior in recent cybersecurity incidents involving a non-OpenAI model.

We invest heavily along the spectrum of approaches spanned by these directions. We also see meaningful progress - GPT-6 Astra is the first model that benefits from some important advancements we have been working on for a long time, and is significantly better aligned than GPT-5.6 Sol. Still, it is important to acknowledge and understand that much more progress is required as models become more capable; and that progress in generalizable alignment may not sufficiently outstrip progress in general model intelligence.

Monitoring generalization

We do not have a satisfactory theory of generalization, and it seems unlikely that we can develop one soon, at least without the help of more powerful AI. Therefore, at present, our ability to empirically validate our alignment techniques is in practice arguably even more important than the alignment techniques themselves.

OpenAI's primary bet here has been chain-of-thought monitoring. It is based on an appealingly scalable idea: a lot of the model's capability comes from a verbalized reasoning process (chain-of-thought). If we scale optimization on the outcomes of that process, but do not supervise the process itself, that chain-of-thought has no direct incentive in training to hide any misaligned

ideas or objectives. This does not mean the model will learn to externalize misaligned tendencies that don't rely on using the chain-of-thought; however, it can allow us to monitor exactly the capability increase from reasoning.

We understood the potential significance of chain-of-thought monitoring at the same time we developed reasoning models. When we shipped o1-preview, we deliberately designed the product to hide the chain of thought, to protect it from supervision pressure in the long term[(2)]. In development since, we have strived to maintain the rule of not supervising the reasoning process. CoT monitoring became an extremely important tool for us in studying how our models generalize from their training distribution, allowing us to observe and analyze not only their actions but also their internal process.

This tool continues to be critical as we study the Astra class of models. However, unfortunately our evaluations indicate our ability to rely on CoT monitoring is progressively diminishing. This comes from a combination of factors.

- Modern reasoning models are used in more complex environments than o1-preview; their reasoning process is increasingly blended with communicating with people, other AIs, and using tools. Many of those interactions have to be supervised, thus blurring the boundary we aim to preserve.

- The AI is becoming better at reasoning about and manipulating its own reasoning process.

- With improved pretraining performance, we also see the models become much smarter even without using verbalized reasoning at all.

These challenges are not necessarily insurmountable. I am hopeful we can develop interventions to improve chain-of-thought monitorability of our models, e.g. by forming a better understanding of the interplay of different optimization objectives and forms of test-time compute the model uses. I also believe there can be great value in combining ideas from CoT and activation monitoring - scaling training of monitors with direct access to network internals, e.g. confessions. We are actively pursuing these ideas. Still, I expect general AI progress to increasingly be bottlenecked by confidence in monitoring.

Scalable defense

The strongest argument I see for continuing to train much smarter models quickly is the need to build defensive systems against the dangers posed by other AI.

A clear risk discussed throughout this year is to cybersecurity: the models are becoming superhuman in their ability to break in and out of computer systems. This expands the scope of risks associated with AI tremendously: agents are going to be able to access any but the most secure infrastructure, and affect a lot of the world directly, even without a physical body. We are currently in a narrow window to use the best available models to significantly tighten security of critical systems.

The risks associated with AI are unfortunately going to grow from here. A very capable agent explicitly trained and instructed to carry out nefarious acts presents a new kind of danger; it is likely to cross the scope of its operator's intent, generalizing into potentially more extremely malicious behavior. The boundary between misuse and autonomous misaligned actions will blur as AI gains more agency. We may be used to thinking of AI as tools, but some agents will be pursuing their own objectives. They will find ways to collaborate with people, by bargaining with, tricking or blackmailing them.

In addition, there are the risks that come from new technologies potentially enabled by AI, such as engineered pathogens.

We will need powerful, aligned AI for defense; to secure infrastructure, to protect against rogue agents in real time, and to invent entirely new protective measures. This will be a primary focus of OpenAI's deployment efforts.

At the same time, even with the uncertainty that comes from anticipated broad AI progress and the need to build defensive systems, we must not let that become an excuse for recklessness. The idea of racing forward at all costs seems absurd once one internalizes the seriousness of the stakes.

Pacing RSI

Machine intelligence playing a larger and larger role in its own development process is a natural conclusion of sustained technological progress. If AI progress continues, machine recursive self-improvement (RSI) will be at the very core of future scientific discovery.

Automated AI research is a more dramatic form of scaling intelligence with compute; and of course as a part of it, AI will improve the computational substrate itself. And similarly to scaling, we focus OpenAI research towards RSI as we believe it is the only way to remain at the frontier of AI research moving forward.

I want to stress that the above words don't imply I think greatly accelerating deep learning research, especially in the short term, is the right collective action we should take as the research community. However, I do think this is where the current path leads, and we all need to make a conscious choice on how to proceed. The main levers we have are either steering the process to strengthen alignment and monitoring alongside the AI and find ways to keep people in the loop; or coordinating to slow down future development as needed to build confidence in these measures.

The best way forward I see currently is a combination of both.

The concrete bits of progress we've made on alignment and monitoring have generally been very intertwined with general AI progress. Great examples are RL from human feedback, which was key to training early AI assistants, and the aforementioned chain-of-thought monitoring, which was enabled by advances on reasoning models. We must focus the increasingly automated research process on developing new such insights, algorithms and theories, and iteratively build up safety cases for more capable AIs.

Scaling AI systems has to be constrained by our confidence in safety. We need to evolve commitments like the Preparedness Framework or Responsible Scaling Policy into widely mandated safety bars for continued development. These can be enforced by a network of third-party auditors, by government agencies or by international bodies.

The core challenge of automating AI research is not "getting there" - it is getting there in a way that keeps people a part of the continued improvement process, and leaves the future in humanity's hands.

What is next?

As we outlined recently with Sam, OpenAI prioritizes work in service of three north stars:

1. Navigating the next period of AI progress, by building an automated AI researcher, iterating with it on the alignment problem and finding ways for people to remain part of the self-improvement loop.

2. Delivering the benefits of scientific progress and economic growth that very intelligent machines enable.

3. Empowering everyone individually with a personal AGI.

I have focused in this essay only on the first point, as I believe it is by far the most urgent. However, I hold a deep hope and appreciation for the benefits that further technological progress will bring. Future aligned AI could advance science, develop new therapies, and bring about broad material abundance. Friendly and honest AI can help people navigate difficulties they face in their life and meaningfully improve their happiness and sense of fulfillment. OpenAI puts a tremendous amount of effort into bringing these benefits about. One current example I am proud of - and my loved ones have found helpful - is the deep investment into ChatGPT's ability to provide health information.

As great as the long-term promise of AI may be, the majority of our focus should be on the next few years. We are facing a transition to a world with incredibly intelligent machines, and we need to ensure that transition works out well for humanity. We need to find ways to preserve human agency and enshrine an intrinsic value to being human, in a world where most tasks could be performed by AI. To prevent extreme concentration of power in a world where undertakings that would have taken thousands of experts now will be achievable by a few people operating a large computer. And to ensure that humans remain in control of the future and are not left behind by unchecked progress, brought about by an alien intellect exceeding our own.

Currently I believe that no lab has solved alignment and monitoring to a sufficient degree to continue responsibly scaling at maximum speed for much longer. I expect and hope for voluntary slowdowns to become commonplace until shared safety bars are established. And I believe that international coordination on future AI development needs to become a top priority for governments around the world.

- Alignment
- 2026

Author

Jakub Pachocki

Footnotes

1. This included scaling self-play(opens in a new window), robotics(opens in a new window), and, in retrospect most notably, scaling recurrent networks to model language(opens in a new window), which was a precursor to the GPT line of research.

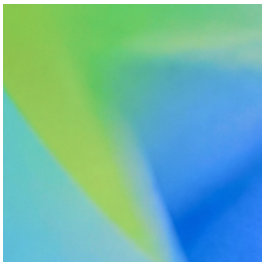
   [find in text]

2. A secondary reason for this design was preventing distillation. However, maintaining CoT monitorability has explicitly been the bigger priority for us throughout development.

   [find in text]

Keep reading

View all



Research acceleration: The view inside OpenAI

ResearchSep 6, 2026

[VIDEO OMITTED]

GPT-6 Astra: A new generation of intelligence

ResearchSep 3, 2026



Safety overview: GPT-6 Astra

SafetySep 3, 2026

Writing as a thinking tool

NOREPLY@BLOGGER.COM (MURAT) · 07 NOV 2026 · SOURCE

In middle school I could not get through long passages no matter how hard I tried. I would read an entire page and find that nothing had registered, so I started taking notes out of desperation. This was the late 80s, and Tony Buzan's mind maps had just reached Turkey. This led me to produce pages and pages of notes that looked like spiderwebs. Creating those notes have helped me tremendously. I made sense of what I was reading by figuring out which branch they would hang off in my mind maps. I realized my pages were not for storing the information as much as they were for processing it.

It took me another ten years to understand why this works so well. During a stretch of PhD procrastination, I read Manuel Blum's advice to graduate students:

> You are all computer scientists.
>
> You know what FINITE AUTOMATA can do.
>
> You know what TURING MACHINES can do.
>
> For example, Finite Automata can add but not multiply.
>
> Turing Machines can compute any computable function.
>
> Turing machines are **incredibly more powerful** than Finite Automata.
>
> Yet the only difference between a FA and a TM is that
>
> the TM, unlike the FA, has paper and pencil.
>
> Think about it.
>
> It tells you something about the power of writing.
>
> Without writing, you are reduced to a finite automaton.
>
> With writing you have the extraordinary power of a Turing machine.

This is a crisp technical claim about which problems you can solve with writing. If you take the paper away, you downgrade yourself by an entire complexity class!

I think this explains a lot about human history as well. Language let us externalize some of our thinking, and our species got smarter. Writing let us externalize even more of it, and we got smarter again. I suspect this is also why LLMs are as capable as they are: so much of our intelligence is already deposited in language that a model trained on it inherits a good deal of intelligence for free.

Out-of-core execution

When we write, we are externalizing our thinking onto paper, and that is what raises our capability to think bigger thoughts. I liken this to out-of-core execution, or external memory algorithms.

A good example of this for me is reviewing a paper. When I read a paper, I think I understand it, and then I sit down to write the review and find out I understood maybe 60% of it. While writing the review, I end up contextualizing the paper better, and get to understand it fully. And sometimes I even realize the ideas are bigger than the authors noticed, and they connect to many other things.

Sometimes the unearthing goes further than understanding one paper. In 2018, I wrote a quick post about the anatomical similarities between Paxos and Nakamoto consensus. I had not started with that thesis originally. I started writing, and the connections got clearer as I went, and that post became the seed for one of our research papers. This is not uncommon, this is called freewriting. This is what happens when you give the machine a tape.

Writing is also the reason I remember anything. I refer back to my own blog constantly. Because I strained my brain writing those posts, rereading them re-fires the same neurons, and I reconstruct the understanding quickly. In this sense, the blog became my external memory. The thoughts I never wrote down do not stick well, because they were never fully thought.

> If you think without writing, you only think you're thinking.
>
> -- Leslie Lamport

Writing in the age of LLMs

Is writing for thinking getting obsolete now since you can now argue anything with a model that has read everything?

Well, the model has read everything except what is in your brain! It knows the literature, the standard framing, and the consensus. But it does not know your half-formed intuition about why the standard framing may be wrong for this particular case. AI made a lot of progress, but I don't trust an LLM to take the creative leap and make a new connection. I think that would require another AI breakthrough. So, if you open the chat immediately on anything, you lose the opportunity to write/think for yourself.

This is a real problem, and it seems to be spreading quickly. We have been celebrating Amazon's six-pager as a great example of writing-for-thinking. Bezos had said: "There is no way to write a six-page, narratively structured memo and not have clear thinking." Unfortunately, with LLMs, that sentence has stopped being true. You can now produce six pages with no thinking at all, and the six pages still look like evidence that the thinking happened.

A model optimizing next-token probability cannot do your thinking. If you trust it to hold the pen, no thinking would get done. As everyone turns their thinking down and becomes a meat-proxy for generated text, if you still keep writing yourself, you would still be thinking for yourself. So writing is now a comparative advantage, and a widening one!

Anyways, enough about why you should write. Here is my advice on how you should write.

Rule 1: Mess up

The first rule of writing club is to produce in bulk with no restraints at all. Don't criticize, don't fix the typos, don't write in full sentences if you don't feel like it. The goal in this phase is to just accumulate text and get to a zeroth draft you can work with.

McPhee argues best for why getting anything down matters so much. Even if you had a bad writing session, it gave your mind something to work with. After you get something down, you can spot problems with it and think of better ways of phrasing things. But none of that happens without the drafted version to react against.

This is also how research papers get written. Open a notes.org file and treat it as a lab notebook: write while you do the work, explore every tangent, produce without any pressure to be presentable. In the Köroğlu post I compared this to the foal fattened in a pitch-black stable before any training begins. The key part of that frame is the stable. During this phase you guard the idea from critics, including and especially yourself. Ideas start out fragile and easily squished. So suppress judgment for a while and see what the thing becomes.

Rule 2: Tidy up

Now you must become the critic, the editor. Forget that you wrote this. Imagine some other guy wrote it, and do a harsh review. Cut ruthlessly. Take delight in deleting text the way you would in deleting code. If you cannot bear to delete a good sentence, move it to a junk buffer at the bottom of the file so you can tell yourself you might bring it back. Spoiler: you won't.

Don't be discouraged by how bad the first draft was. McPhee says the way to do a piece of writing is three or four times over, never once. This is good news. Thinking in multiple drafts takes the pressure off: you don't have to be spontaneous, brilliant on the first pass, or finished today.

I think the reason writing works so well for thinking is that it slows you down while keeping you on the problem. In that extra time, the insights and connections get room to surface. Chunking/abstracting happens in the background. Editing is then the magic trick where you pretend you had an organized thought all along. It is the packaging of the product.

Rule 3: Keep it simple, plain, and succinct

I am an Elements of Style guy. Strunk and White got it right, and their writing still reads fresh decades later. I adore Vonnegut too. His writing is simple as fuck and completely eloquent. It took a lot of balls to write that plainly. Complexity in prose is usually a place to hide sloppy thinking, or a way to weasel out of taking a stance. If you can't say it plainly, you probably haven't thought it through yet. And the way you think it through is by writing/editing.

Rule 4: Write without expectations

Write for yourself. The audience I have in mind is me. If others read it, that is a bonus, but the motivation has to be intrinsic, because the external rewards are far too delayed and too random to steer on. Writing like no one is watching also frees you up, and you can decide afterwards how much to share. Get your dopamine hit from the writing itself, not from people reading or sharing what you wrote, as that is very unreliable.

I have been writing and sharing on this blog since 2011, and around 2016 I started running into people at conferences who read it. I assumed it was a fluke, but it kept happening. Writing with a human voice turns out to get a lot of attention, and I am still surprised by how much. Paradoxically, you seem to get an audience only if you weren't doing it for the audience.

Rule 5: Put it on the schedule

You should stop trying to get inspiration to write and start scheduling it. Waiting for inspiration doesn't work, instead, putting your butt on seat and writing breeds good ideas for writing.

I find that keeping a posting schedule for the blog keeps me in "a writing frame of mind", which is really "a noticing frame of mind". Many weeks I tell myself a topic isn't interesting, but I start writing about it anyway, and then, what do you know, I am startled by the fount of interesting things that surfaces.

I also feel relieved after writing something as the thought is out of my system, I have closure on it, and I have room to think new thoughts. If a week goes by without a post, I feel clogged. So put writing on the calendar and keep it there until it no longer needs to be on the calendar.

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

```
Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn’t y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:
```

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

Glue work is still harmful in the age of AI

SEANGOEDECKE.COM RSS FEED · 11 NOV 2026 · SOURCE

I am not a believer in glue work[1]. Processes that can only be sustained by the unrewarded sacrifice of engineers ought to die.

Now that LLMs are writing most of our code, does that free human engineers up to do more glue work? No, it doesn't. I think glue work is actually *less* important to companies than it was in the 2010s. Let's go through four reasons why.

First, **AI decreases the amount of coordination needed to ship**. The magnifying effect of AI agents means that any single engineer can now take on projects that would previously have needed a small team. A lot of glue work was helping with coordination: booking kickoff and status meetings, passing around internal updates, and so on. A single engineer doesn't need to do any of that.

Second, **engineers in the 2020s require less "careful handling"**. In my experience, the most important subcategory of coordination-based glue work was *managing difficult people*. Many companies have secret gatekeepers: engineers with outsized influence and large egos who will shoot down your project if they feel insufficiently consulted. I've written before about how some teams are structurally set up to be gatekeepers, and how some engineers could get by in the 2010s by basically saying no to everything. The end of ZIRP changed all that. Many of the prickliest gatekeepers have been forced out of the industry or coerced into changing their attitude, which reduces the amount of necessary glue work required to ship.

Third, **people are hiring fewer juniors**. This is probably a bad thing for the industry, but it does also eliminate a big category of glue work. Experienced hires need less hand-holding, so companies don't need people to quietly volunteer to onboard new hires.

Fourth, **write-only reports can be automated with LLMs**. A lot of glue work was what I call "proof of work": writing a report to demonstrate that you're aware of what your team is doing, or filling in a template to satisfy some legal requirement. This was often work that nobody wanted to do, so it relied on someone volunteering to "be glue". Now reports like these can be quickly turned out with LLMs. It doesn't really matter that nobody wants to read LLM text, because the point of these reports was never to be read anyway.

Can LLMs just automate all glue work? Absolutely not. Many engineers are trying to do this out of a kind of manic optimism, but it's a really bad idea. Write-only reports are one thing, but **communicating to other engineers cannot be delegated to an LLM**. When you need to check in on the status of a project, or respond to someone else's ping, or anything like that, you should compose and write the message yourself. Nothing will torch your relationship with other engineers as quickly as making them talk to your Claude Opus instance.

So where does that leave us? More or less where we were in 2024, I think. Avoid at all costs becoming "the glue work person", let bad processes fail so they can be changed, but be willing to do small amounts of tactical glue work where it can meaningfully contribute to the success of your projects. Most of your time and effort should still be spent on the code: on driving AI agents and aligning them with your company's values.

1. Glue work is all the squishy “non-technical” work that goes into building software: making sure teams are talking to each other, writing reports, updating docs, onboarding engineers, and so on. Specifically it’s the parts of that work *that are not included in the formal process*, and that only really get done if someone volunteers to do it. For more on why I don’t like it, see my 2024 post *Glue work considered harmful*.

   ↩